# IndiskaAI Technical Assessment Submission

## Deliverable 1: Project Analysis Report

1. High-Level Overview

In computational biotherapeutics, an antibody consists of two interacting protein chains: a **Heavy Chain** ($V_H$) and a **Light Chain** ($V_L$). During drug screening and development, scientists typically prioritize optimizing the heavy chain because it contributes the majority of antigen-binding specificity. However, an isolated heavy chain cannot fold properly or function as a viable drug without a compatible light chain.

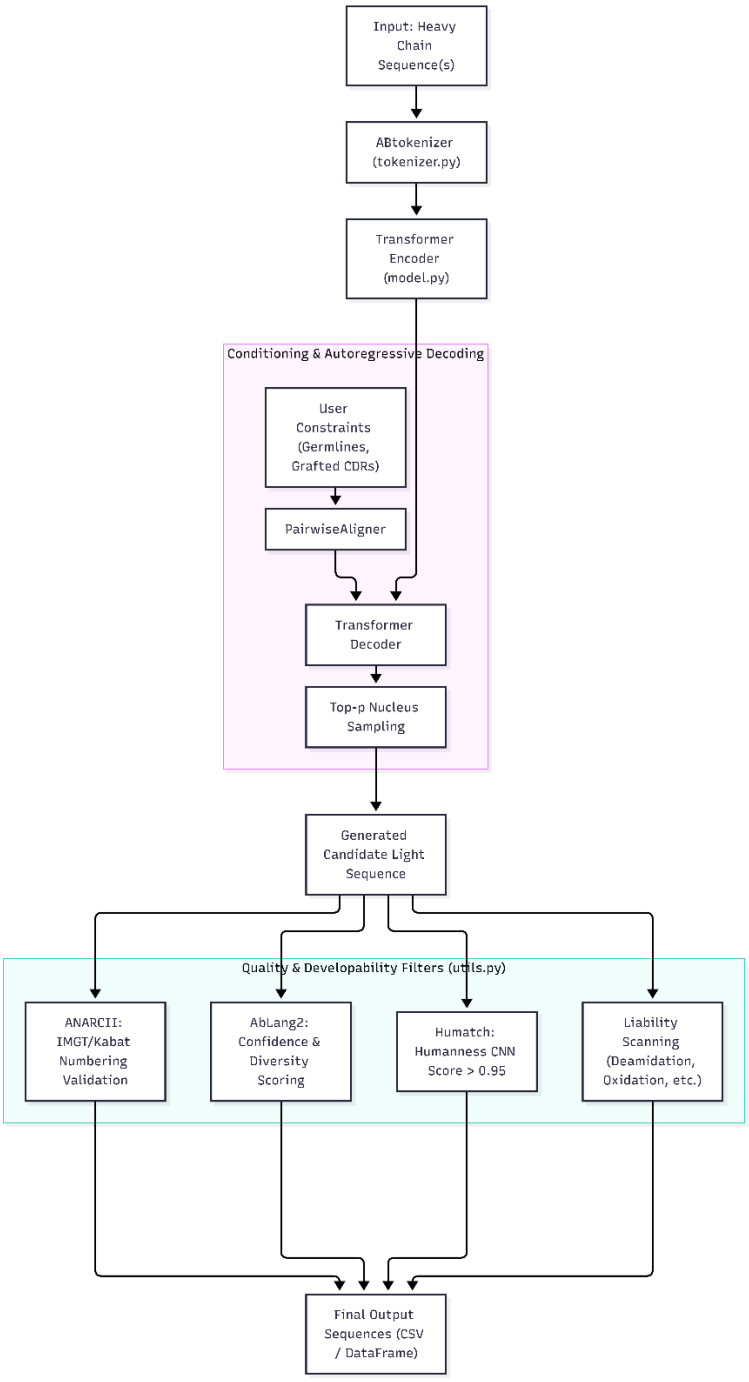
Experimentally discovering or matching a compatible light chain in a wet lab requires months of screening. Furthermore, in **bispecific antibodies** (therapeutic molecules engineered to bind two distinct disease targets at once), two different heavy chains must share a single "common light chain" without mispairing.

**LICHEN** (**L**ight-chain **I**mmunoglobulin sequence generation **C**onditioned on the **H**eavy chain and **E**xperimental **N**eeds) addresses this challenge using deep learning. It frames antibody pairing as an **NLP sequence-to-sequence translation task**:

- **Input:** A Heavy chain amino acid sequence (or two heavy chains in bispecific design).
- **Output:** A compatible, biologically viable Light chain sequence.
- **Experimental Conditioning:** Allows researchers to specify design constraints, such as preferred gene families, chain types ($\kappa$ or $\lambda$), or grafting pre-determined Complementarity-Determining Regions (CDRs).

2. Architecture and Workflow Explanation

LICHEN combines a deep generative Transformer with heuristic sequence alignment and multi-tiered downstream filtering.




Workflow diagram

Workflow Stages:

1. **Tokenization:** Amino acid sequences are tokenized using a 24-character vocabulary: 20 standard amino acids plus 4 structural control tokens:
   (< start, > end, - padding, * wildcard).
2. **Encoding:** The heavy chain sequence is processed by a 6-layer PyTorch Transformer Encoder with sinusoidal positional embeddings ($d_{\text{model}} = 512, h = 8$).
3. **Bispecific Dual-Conditioning:** When two heavy chains are provided for bispecific design, both are encoded independently into memory tensors ($M_1$ and $M_2$). At every decoding step, predicted next-token probability distributions from both representations are averaged
4. **Constrained Autoregressive Generation:**
   - **Germline/Seed Conditioning:** Injects specified N-terminal framework tokens at the start of decoding.
   - **CDR Grafting:** Uses Biopython's *PairwiseAligner* with a BLOSUM62 matrix to match user-provided CDR loops against reference framework starts. Conserved structural anchors (Cys23, Trp41, Cys104) are monitored during generation, and specified CDR segments are spliced into the sequence dynamically.
5. **Quality Filtering Pipeline:** Candidate sequences are filtered against structural numbering viability (ANARCII), human-likeness score (Humatch), and natural sequence confidence/diversity (AbLang2).

3. Key Components and Code Responsibilities

| File | Module / Class | Primary Responsibility |
|---|---|---|
| *model.py* | Seq2SeqTransformer, PositionalEncoding | Pure PyTorch neural network architecture: 6 encoder layers, 6 decoder layers ($d_{\text{model}} = 512, d_{\text{ff}} = 512, h = 8$). |
| *inference.py* | Heavy2Light | Autoregressive decoding loop, top-$p$ nucleus sampling, bispecific dual-memory averaging, and log-likelihood/perplexity calculations. |
| *pretrained.py* | LICHEN | High-level user-facing API (light_generation, light_generation_bulk, light_log_likelihood, light_perplexity). |
| *tokenizer.py* | ABtokenizer | Bi-directional mapping between amino acid characters and PyTorch tensor indices. |

| File | Module / Class | Primary Responsibility |
|---|---|---|
| *load_model.py* | load_model, configure_device | Weights loading (torch.load), hardware detection (GPU/CPU), and thread allocation via psutil. |
| *utils.py* | FILTERING, get_sequence_liabilities | External tool wrappers for ANARCII, Humatch, and AbLang2; regex-based chemical liability scanner. |
| *cli.py* | main | Command-line interface with argparse for running bulk inference from .fasta or .csv files. |

4. Dependencies and Technologies Used

- **Deep Learning Framework:** PyTorch (*torch.nn, torch.optim*)
- **Bioinformatics & Structural**
  **Libraries:** Biopython (*Bio.Align.PairwiseAligner*), ANARCII (Hidden Markov Model antibody numbering), AbLang2 (antibody language model), Humatch (CNN humanness classifier)
- **Data Engineering & System:** pandas, numpy, psutil
- **Packaging Standard:** hatchling (modern packaging standard)

5. Potential Limitations, Bottlenecks, and Challenges

1. **Manual Weights Distribution:** The large model weights file (model_weights.pt) is hosted on Zenodo and omitted from GitHub, causing immediate runtime errors for new users if not manually placed.
2. **Eager Coupling of Auxiliary Dependencies:** In the original codebase, initializing the LICHEN model unconditionally instantiated FILTERING, forcing imports of anarcii and ablang2 even when a user only requested basic sequence generation.
3. **Absence of KV-Caching in Generation:** The decoder recomputes cross-attention across all past tokens on every forward step, creating an $O(N^2)$ computational bottleneck.
4. **Heuristic Hardcoded Anchors:** CDR grafting depends on fixed offsets and expected invariant residues (e.g. Cys23, Trp41), which can fail on engineered or non-standard framework topologies.
5. **No 3D Interface Modeling:** The model works in 1D sequence space without explicitly calculating 3D atomic packing or steric clashes at the $V_H$-$V_L$ interface.

## Deliverable 2: Improvement Suggestions

1. Automated Model Weights Downloader & Integrity Verification

- **Problem:** Model weights (model_weights.pt) are not packaged with the repository. Users must manually look up Zenodo records, download Model.zip, extract it, and place it in the correct relative directory. Missing weights result in an immediate FileNotFoundError.
- **Justification:** Automating the retrieval and checksum verification of pretrained model weights removes friction for new developers, ensures reproducible environments, and enables automated CI/CD and integration testing pipelines without manual human intervention.

2. Lazy Loading of Heavy Auxiliary Dependencies

- **Problem:** In the original design, *LICHEN.__init__* unconditionally initialized *self.FILTERING = FILTERING(...)*, eagerly importing and instantiating both ANARCII and AbLang2.
- **Justification:** Many users only require standard unconstrained generation, log-likelihood evaluation, or perplexity scoring. Eagerly loading auxiliary models consumes several hundred MBs of RAM unnecessarily and causes runtime crashes if optional filtering dependencies are not installed on the system. Decoupling filtering through on-demand lazy loading makes the core package lightweight and resilient.

3. Correction of IMGT Coordinate Variable Overwrite in utils.py

- **Problem:** In *utils.py*, the variable IMGT_CDR2 was defined and then immediately overwritten on the next line with the CDR3 coordinate range (range(105, 117+1)), resulting in IMGT_CDRs = [IMGT_CDR1, IMGT_CDR2, IMGT_CDR2].
- **Justification:** This variable overwrite caused CDR-L2 positions (residues 56–65) to be omitted from liability scanning, while CDR-L3 was scanned twice. Correcting this ensures accurate biophysical liability reporting across all three antigen-binding loops.

4. Batched Inference Support for High-Throughput Screening

- **Problem:** The bulk generation method (*light_generation_bulk*) processes input heavy sequences sequentially using a standard Python for loop over rows.
- **Justification:** In real-world drug discovery, screening libraries often contain thousands of antibody candidates. Implementing tensor-level batching allows GPU and multi-core CPU parallelism, reducing generation latency across large datasets.

5. Modernization of Deprecated Biopython Aligner Attributes

- **Problem:** In *inference.py*, the aligner configuration references *end_open_gap_score* and *end_extend_gap_score*, which trigger deprecation warnings in current versions of Biopython (open_end_gap_score and extend_end_gap_score).
- **Justification:** Deprecated attributes are slated for removal in future Biopython releases. Updating to current attribute naming preserves forward compatibility and keeps logs clean from deprecation notices.

6. Addition of Comprehensive Unit Testing & CI/CD Pipeline

- **Recommendation**: Add a *tests/* directory with pytest test suites covering:
    - Tokenizer encoding/decoding roundtrip integrity.
    - Output sequence length and character validity (20 canonical amino acids).
    - Bispecific input handling (verifying that dual inputs don't throw dimension mismatch errors).
    - Path handling on Windows, Linux, and macOS.
- **Justification**: The repository currently contains zero test files. Automated tests ensure regressions are caught early and provide confidence when making code modifications.

## Deliverable 3: Implementation of Selected Improvements

1. **Implementation 1: Automated Weights Downloader (*src/lichen/downloader.py*)**

- Created a dedicated downloader module that checks for local weights, downloads Model.zip from Zenodo if missing, verifies the MD5 checksum to ensure data integrity, extracts the .pt weights, and cleans up temporary archives
- Reason: In industry -standard data science packages (e.g., torchvision, huggingface, spacy), models should be usable immediately upon installation. Forcing manual Zenodo lookups creates friction, breaks headless Docker or CI/CD deployments, and leads to common user configuration errors. The MD5 checksum ensures that corrupted or partially downloaded model files are detected before causing silent inference failures.

2. **Lazy Loading of Filtering Suite (*src/lichen/pretrained.py*)**

- Updated the LICHEN class so FILTERING is only instantiated on demand
- Reason: In production workflows, memory efficiency and modularity are essential. Many production pipelines use LICHEN strictly for sequence generation or perplexity scoring without needing ANARCII or AbLang2 filtering. Lazy loading prevents allocating unnecessary memory, accelerates model startup time, and ensures that the core generation functionality continues to work even in lean deployment environments where heavy auxiliary dependencies are omitted.

## Deliverable 4: Documentation

1. Setup Instructions

**Prerequisites**

- Python 3.10, 3.11, or 3.12 (Python 3.12 verified)
- Git

**Installation**

```
# 1. Clone repository
git clone https://github.com/oxpig/LICHEN.git
cd LICHEN
# 2. Install primary dependencies
pip install biopython pandas torch
# 3. Install LICHEN package in editable mode
pip install -e . --no-deps
# 4. (Optional) Install filtering dependencies if ANARCII/AbLang2 filtering is needed
pip install anarcii ablang2
```

**Execution Options**

- **Zero-Configuration Python Execution:**

```
from lichen import LICHEN
# Weights are automatically detected or downloaded on first run!
model = LICHEN(cpu=True)
heavy =
"EVQLLESGGEVKKPGASVKVSCRASGYTFRNYGLTWVRQAPGQGLEWMGWISAYNG
NTNYAQKFQGRVTLTTDTSTSTAYMELRSLRSDDTAVYFCARDVPGHGAAFMDVWGTG
TTVTVSS"
print(model.light_generation(heavy, n=1))
```

- **Interactive Notebook:** Open notebooks/examples.ipynb and run all cells. All outputs will execute sequentially.
- **CLI Execution (in Powershell):**

lichen
"EVQLLESGGEVKKPGASVKVSCRASGYTFRNYGLTWVRQAPGQGLEWMGWISAYNG
NTNYAQKFQGRVTLTTDTSTSTAYMELRSLRSDDTAVYFCARDVPGHGAAFMDVWGTG
TTVTVSS" -c True -o output.csv -v

2. Assumptions Made During Analysis

1. **Biological Equivalence:** Without prior bioinformatics expertise, antibody sequences were conceptualized as strings over a 20-character biological alphabet governed by structural grammar rules and variable loops.
2. **Domain Lengths:** Assumed input sequences correspond to antibody variable heavy domains ($V_H$, typically 115–125 amino acids) rather than intact multi-domain antibodies.
3. **Execution Environment:** Focused primarily on CPU execution (cpu=True) to ensure reproducibility across standard consumer computing setups without requiring specialized CUDA GPUs.

3. Challenges Encountered & Candidate Resolutions

| Challenge Encountered | Practical Context & Impact | Resolution Taken |
|---|---|---|
| **Domain Terminology Barrier** | As a fresher with no bioinformatics background, specialized terminology (Heavy/Light chains, CDRs, IMGT/Kabat numbering schemes, developability liabilities) was initially unfamiliar. | Mapped concepts to NLP equivalents: Heavy chain = Input Prompt, Light chain = Target Translation, CDRs = Constrained Substring Replacements, ANARCII/Humatch = Rule-based Validation Filters. |
| **Missing Model Weights on First Run** | Cloning the repository and attempting to execute the documented code immediately failed with FileNotFoundError: | Identified the Zenodo record cited in the README, downloaded the archive, and |

| Challenge Encountered | Practical Context & Impact | Resolution Taken |
|---|---|---|
| | model/model_weights.pt because weights are not stored on GitHub. | implemented downloader.py to automate this step for future users. |
| **Tightly Coupled Optional Dependencies** | A simple script for standard generation failed with ImportError: ANARCII is required because the class initialization eagerly loaded optional filtering modules. | Re-engineered LICHEN.FILTERING as an on-demand lazy property in pretrained.py. |

5. Future Enhancement Recommendations

1. **Decoder KV-Caching:** Store key-value projection states during autoregressive generation to accelerate decoding from $O(N^2)$ to $O(N)$ complexity.
2. **True Batched Generation:** Update Seq2SeqTransformer.forward and decode to process batches of heavy chains simultaneously using padding masks.
3. **FASTA File Support in CLI:** Support standard .fasta sequence format inputs and outputs in addition to CSV files.
4. **Streamlit / Web Demo:** Build a lightweight local graphical interface where scientists can input heavy chains and view generated light chains with annotated binding loops.